Normal forms and complete relations for several classes of circuits

Xiaoning Bian

Department of Computer Seience and Technology
Tsinghua University

Presented at Quantum PLT seminar, DCST, Tsinghua U
Oct 16, 2025

Contents

Goals

- An introduction for circuit normal form and circuit rewriting.
- Formalizing some easy cases in Agda.
- A better understanding of quantum circuits.

6 classes of circuits (classical and quantum)

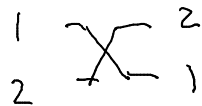
- In 2003, Lafont first gave the theory and serveral examples [2]. We will learn 2 examples from it.
  - Circuits of permutations and even permutations.
  - Circuits of linear permutations and affine permutations.
- In 2015, Selinger gave normal forms and complete relations for Clifford circuits [4].
- In 2019, Amy, Jianxin Chen, and Ross dittoed for CNOT-Dihedral circuits [1].
- In 2024, Bian did the same for $C_m^n \rtimes S_n$ in Agda, unpublished.
- In 2025, Li et al. did the same for qutrit Clifford circuits [3].

Note: this is not a full list of related work.

Permutations — S_2

permutation of two items.

id



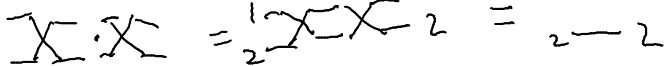
σ

$S_2 \simeq C_2$, cyclic group of order 2.

Consider a circuit consists of σ, id, e.g.

Permutations — S_2

by looking at the action on two items, we know

(order): 

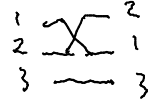
using order rule, cir 1 rewrite to

Permutations — S_2

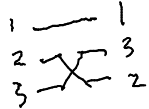
We say $\{\sigma, id\}$ are normal forms for S_2 and $\{XX = \overline{\ }\}$ is a complete set of relations for S_2. It is complete because any other circuit rewrite to a unique normal form using rewrite rules from this set.

Permutations — S_3

S_3 is generated by

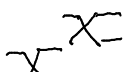


(σ_{12}) ,



(σ_{23})

$|S_3| = 3! = 6$. S_3 has 6 elements:

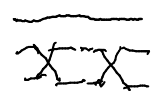
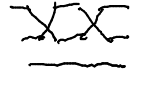
①  , ②  , ③  , ④  ,

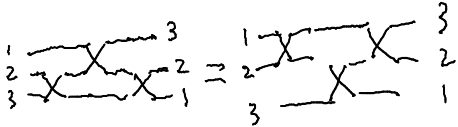
⑤  , ⑥  .

Permutations — S_3

We say S_3 has normal forms $\{①, \cdots ⑥\}$.

Claim: $\{ⓐ, ⓑ, ⓒ\}$ are complete, where.

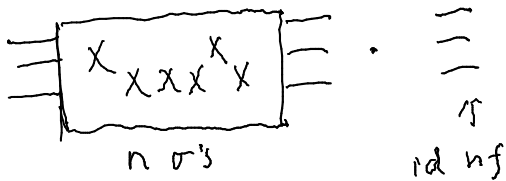
ⓐ  , ⓑ  ,

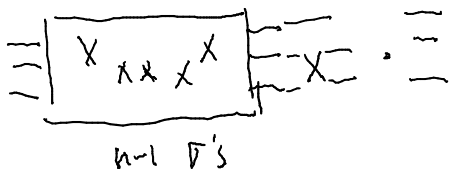
ⓒ 

NOTE: Soundness can be checked by lookily at their actions.

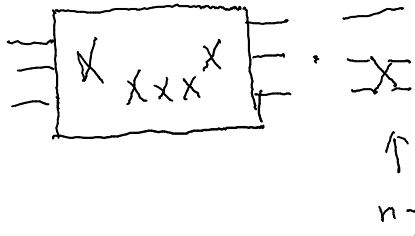
Permutations — S_3

proof : Consider a circuit with n gates.





Permutations — S_3



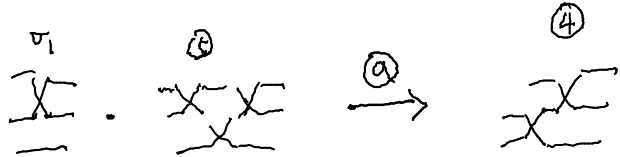
So, essentially we need to show for each

$x \in \{\sigma_1, \sigma_2\}$, and each $y \in \{①, \cdots ⑥\}$, $\exists\, y' \in \{①\cdots⑥\}$

Permutations — S_3

using only rules ⓐ ⓑ ⓒ.

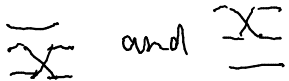
St. $x \cdot y \longrightarrow y'$. There are $2 \cdot 6 = 12$ cases.

e.g. 

e.g. 

etc. So $\{ⓐ, ⓑ, ⓒ\}$ is complete □.

Permutations — S_n

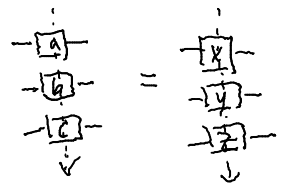
 and are similar, they are built from basic parts — and  , through "vertical composition". Likewise, rule (a) and (b) are similar, they are just order rules.

Permutations — S_n

- This is the main feature of circuit rewriting.

  Compared to string rewriting, where you only have "1-dim" rewriting, e.g. "abcd" → "xyz".

- "vertical structure" is also quite restrictive.

You don't have rule like 

except when you have

the equality from horizontal rewriting.

- But you are allowed to "slide"

This is actually a "structure" rule meaning they are built in, no need to explicitly mention them.

Lafont 2003 formalized this idea using category theory. The vertical composition is a monoidal structure.

Recall Bonan introduced monad last week, and a famous quote "a monad is just a monoid in the category of endofunctors". To define.

Permutations — S_n

"monoid", monoidal structure is needed.

The takeaway is: monoidal, monad, monoid are related to structure of the form

$$\{ A \times A \xrightarrow{\bullet} A \;,\; \Sigma \;,\; \bullet\text{-associative} \;,\; \Sigma\text{-(left/right-)identity} \}$$

Lafont 2003 also has several examples: S_n, A_n ....

Permutations — S_n

S_n, using the vertical composition idea, is a groupoid, generated by . All possibu

↑ something like a group

paddy, on top of, and under , e.g.  , 

are now "built in" through the monoid structure.

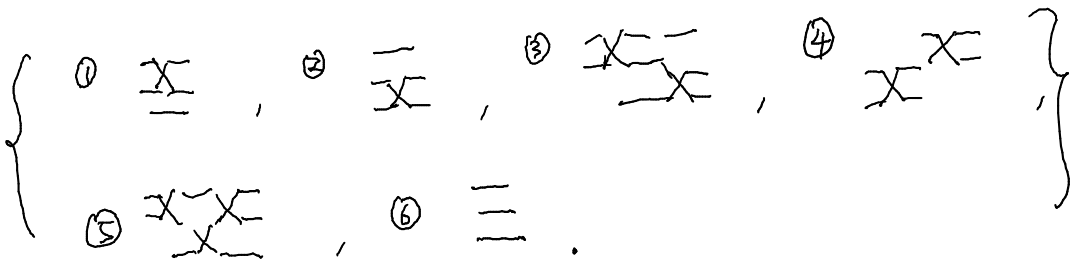
Likewise, the "sliding rule" are built in too.

Permutations — S_n

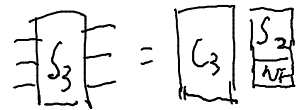
S_n has normal forms (nfs):

Permutations — S_n

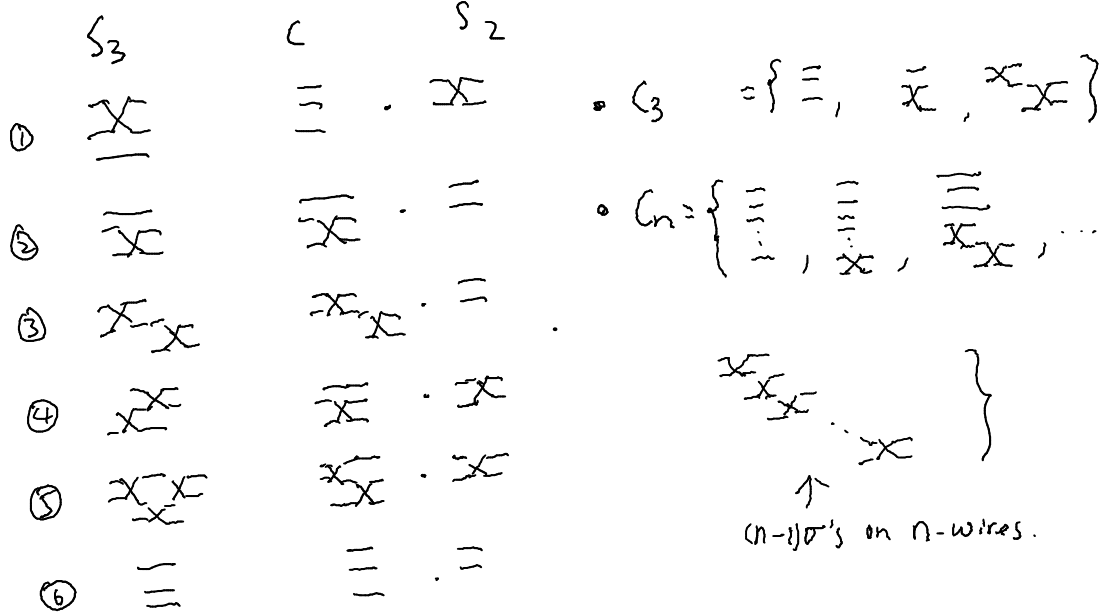
e.g. S_3/S_2 .. $S_2 = \{$  $\}$

$S_3 =$ 

Can you spot the factorization?

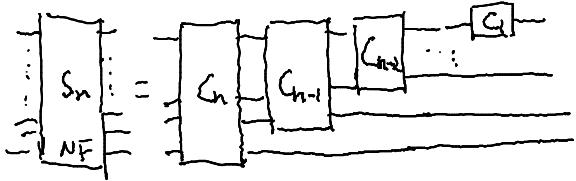


Permutations — S_n



Permutations — S_n

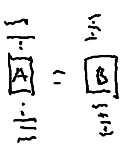
So,



Claim. {  , 

,

,

If $\boxed{A} = \boxed{B}$, then 

} is complete.

Yang-Baxter rule

vcong.

Permutations — S_n

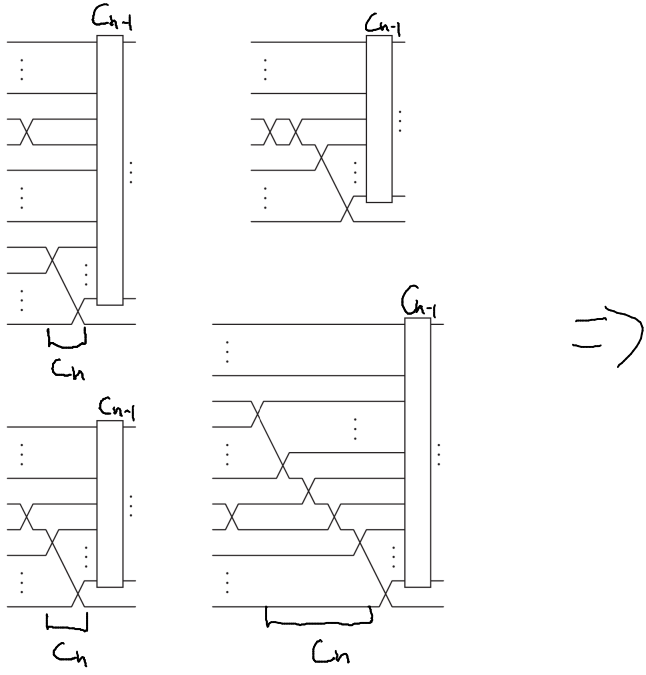
In groupoid setting, no need to mention rule (comm) & (vcong).

Like before. we show for-all σ_i, and all nfs, $\exists$ nf'

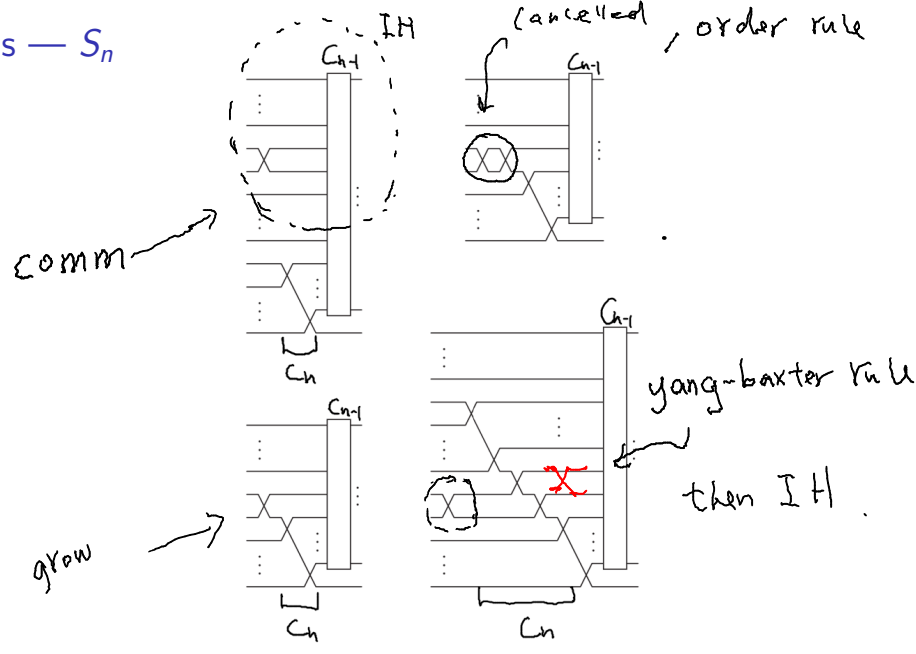
$$\sigma_i \cdot nf \longrightarrow nf'$$

There are 4 cases of this concatenation.

Permutations — S_n



Permutations — S_n

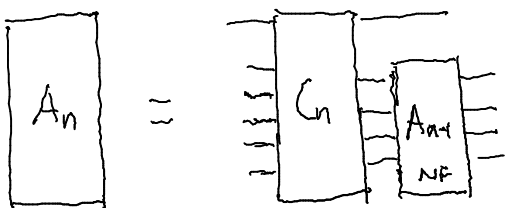


Even permutations — A_n

A_n is generated by $\left\{ \times = \times \right\}$.

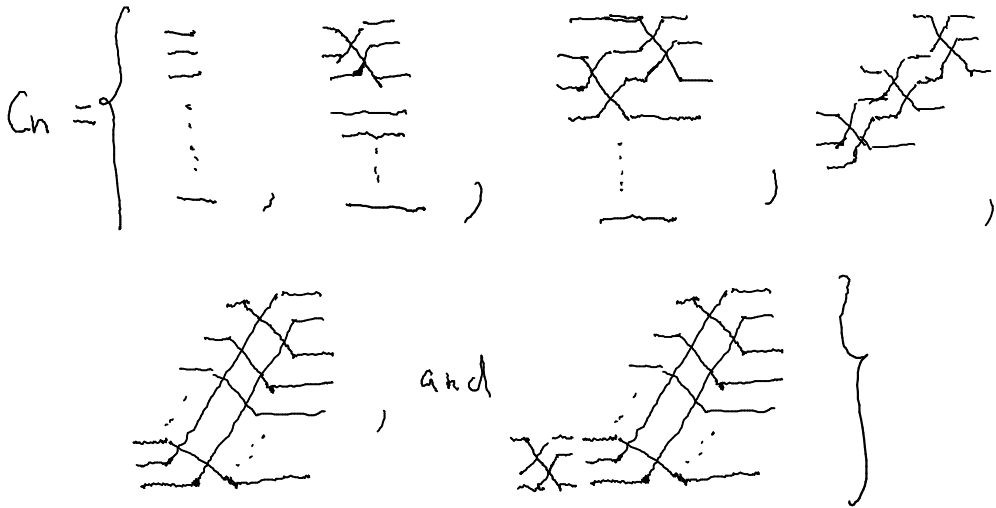
$A_n \leq S_n$ of index 2, i.e. half of S_n are even.

NFs:



, where

Even permutations — A_n

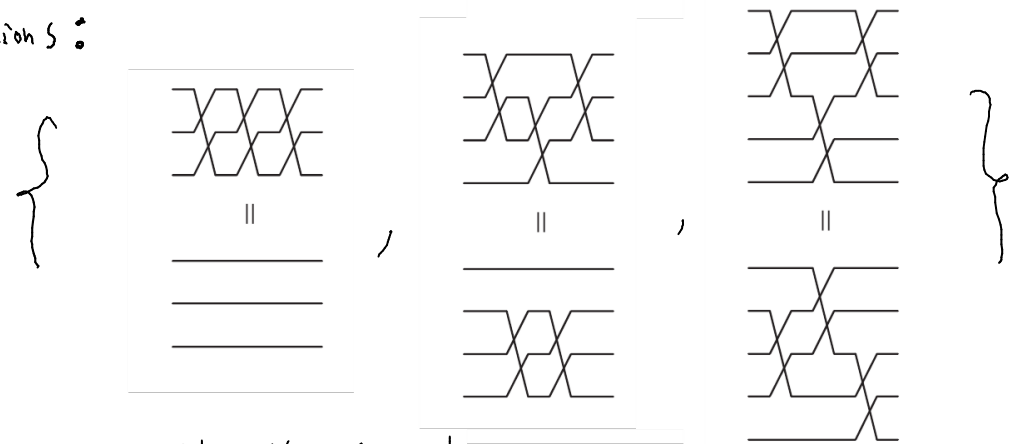


Even permutations — A_n

$|C_n| = 1 + (n-2) + 1 = n$ for $n \geq 3$.

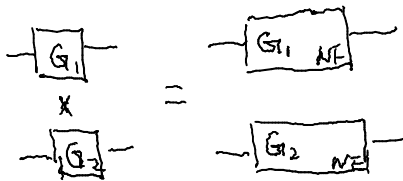
$|A_n| = n!/2$. $|C_2| = 1$.

Complete relations:



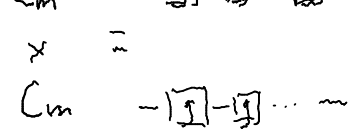
proof of completeness, see Lafont 2003. More complicated than one think.

$C_m^n \rtimes S_n$ — NF construction, direct product, semi-direct product.



e.g. $S_1 \times S_1 = S_1 \wr S_1$

e.g. $C_m \times C_m = C_m \wr S_1$



$C_m^n \rtimes S_n$

Complete relations consists of three parts

(1) rels for G_1

(2) rels for G_2

(3) commuting rules of G_1 and G_2 i.e. sliding.

(implicit in monoidal setting)

$C_m^n \rtimes S_n$

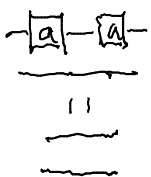
e.g. C_2 generated by a

$\times$

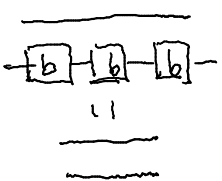
C_3 generated by b

$$\text{rels} = \{ a^2 = e, \; b^3 = e, \; ab = ba \}$$

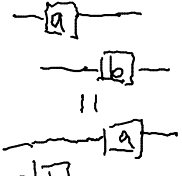
$$= \left\{ \right.$$



,

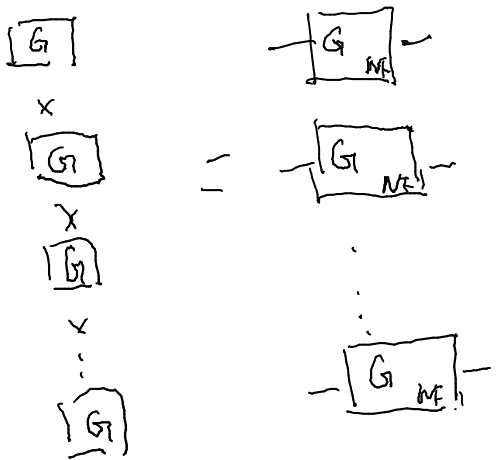


,



$\left. \right\}$

$C_m^n \rtimes S_n$ — NF, Rel Construction — N times direct product.



If we know product of two groups, we know product of N groups by induction.

$C_m^n \rtimes S_n$ — NF, Rel, construction — Semidirect product.

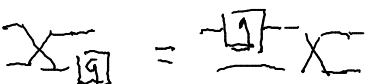
e.g. $C_m^n \rtimes S_n$

↑ ↑

generators $\{$,  $\}$.

relation
1. S_n rels ✓
2. C_m^n rels ✓
3. Semidirect rels ←

and

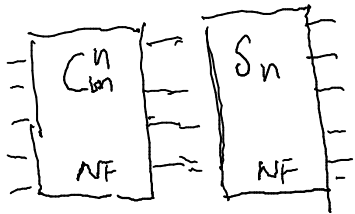


"semi-commute"

$C_m^n \rtimes S_n$

NFs

$C_m^n \rtimes S_n =$



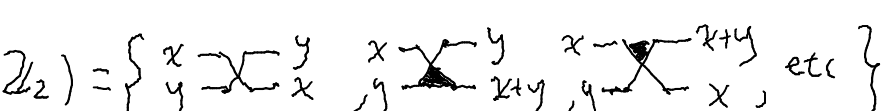
Linear permutations — $GL_n(\mathbb{Z}_2)$

Now, something more interesting.

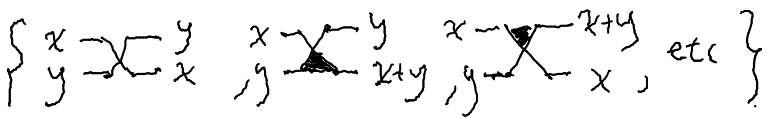
General Linear maps over field $\mathbb{Z}_2$.

$$GL_n(\mathbb{Z}_2) = \left\{ f : \mathbb{Z}_2^n \to \mathbb{Z}_2^n \;\middle|\; f \text{ linear and invertible} \right\}$$

e.g. $GL_1(\mathbb{Z}_2) = \{ id \}$. NOTE: negation is not linear.

e.g. $GL_2(\mathbb{Z}_2) = \{$  $\}$

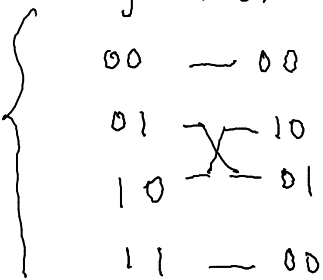
Linear permutations — $GL_n(\mathbb{Z}_2)$

e.g. $GL_2(\mathbb{Z}_2) = \{$ 

$\}$

$xy \longmapsto yx$

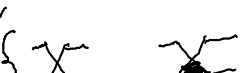
$S_{2^n} \supset$ one particular element of S_{2^n}



Linear permutations — $GL_n(\mathbb{Z}_2)$

 permute bases.

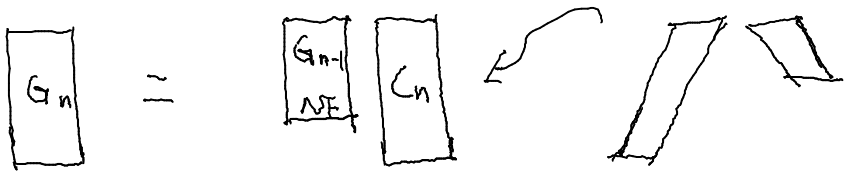
So they are linear & invertible.

Theorem 6. (Lafont 2003). $GL_n(\mathbb{Z}_2)$ is presented by $\left(\{$  $\}$, 6 rels $\right)$.

I did an implementation using Haskell. Demo.

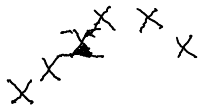
Linear permutations — $GL_n(\mathbb{Z}_2)$

NFs: this time, we use right cosets.

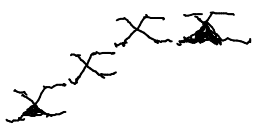


$$|C_n| = 2^{n-1} \cdot \sum_{k=1}^{n-1} 2^k$$

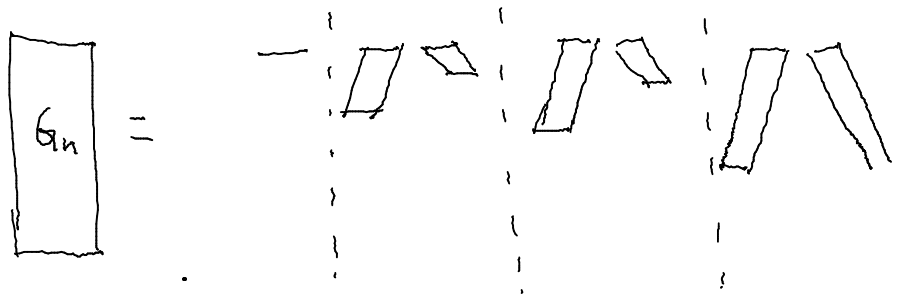
e.g



e.g.

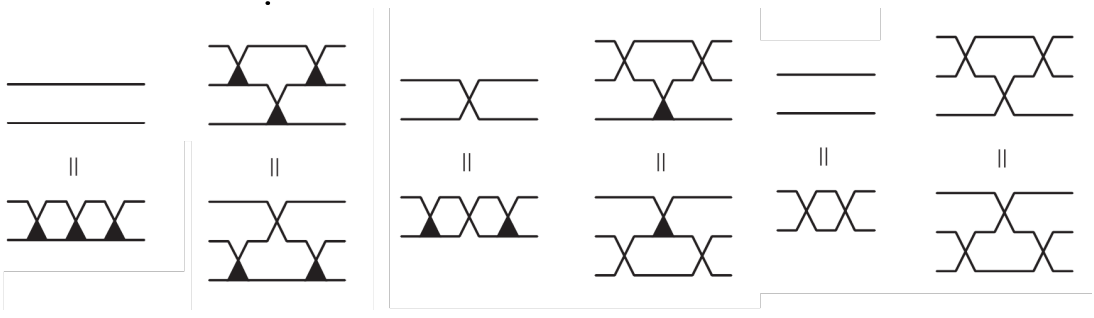


Linear permutations — $GL_n(\mathbb{Z}_2)$



Linear permutations — $GL_n(\mathbb{Z}_2)$

6 rels :



proof, see Lafont 2003.

Affine permutations — $GA_n(\mathbb{Z}_2)$ — adding negation, we get affine.

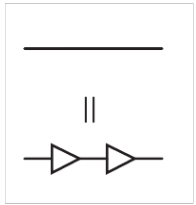
Theorem 11 (Lafont 2003). $GA_n(\mathbb{Z}_2)$ is presented by

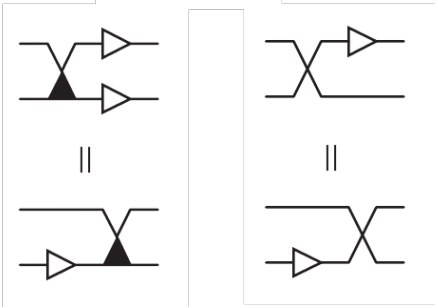
Actually $GA_n(\mathbb{Z}_2) = C_2^n \rtimes GL_n(\mathbb{Z}_2)$, so its rels consists of 3 parts

1. 6 for $GL_n(\mathbb{Z}_2)$
2. 1 for C_2^n
3. 2 for semidirect product.

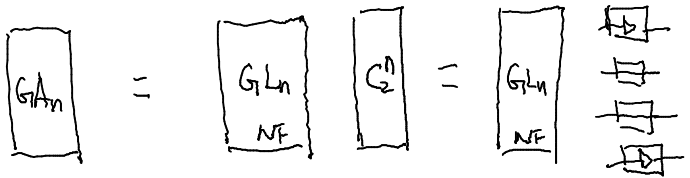
Affine permutations — $GA_n(\mathbb{Z}_2)$

Rels





NFs



CNOT+Dihedral — Now Chen et al.'s work.

CNOT + Dihedral = Circuits over gate set $\{CNOT, X, T\}$.

$\subseteq$ Clifford+T.

- $\{T, X\}$ generates the dihedral group of order 16 mod scalar.

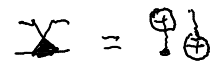
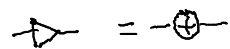
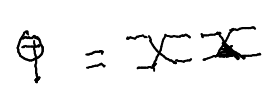
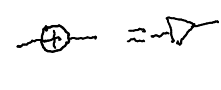
  Complete rels is $\{T^8 = \mathcal{E},\ X^2 = \mathcal{E},\ XT = T^{-1}X\}$.

- In general, Dihedral group of order $2n$ consists of Symmetries of n-polygon.

and $XTX = T^{-1}$

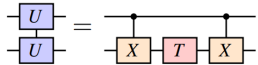
CNOT+Dihedral

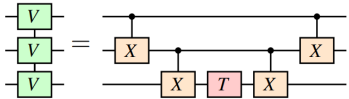
They use two facts:

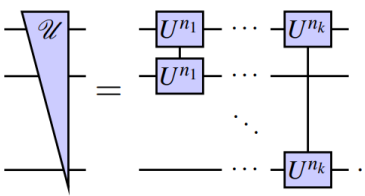
- The group G_1 generated by $\{$  ,  $\}$ is $GA_n(\mathbb{Z}_2)$.
  - To see.  , 
    
  - and.  ,  .
- CNOT+Dihedral $= M \rtimes G_1$, where $M \leq G_8^{2^n}$.

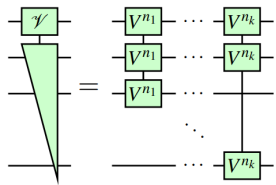
CNOT+Dihedral

NFs: $\mathcal{M}$ NF, $\mathcal{G}\mathcal{A}_n$ NF. For $\mathcal{M}_{NF}$, let

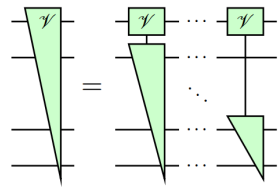




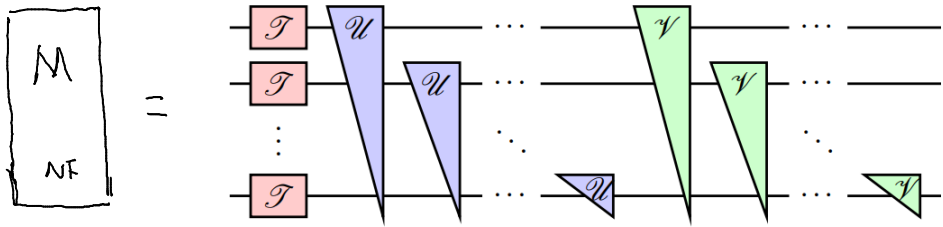




and

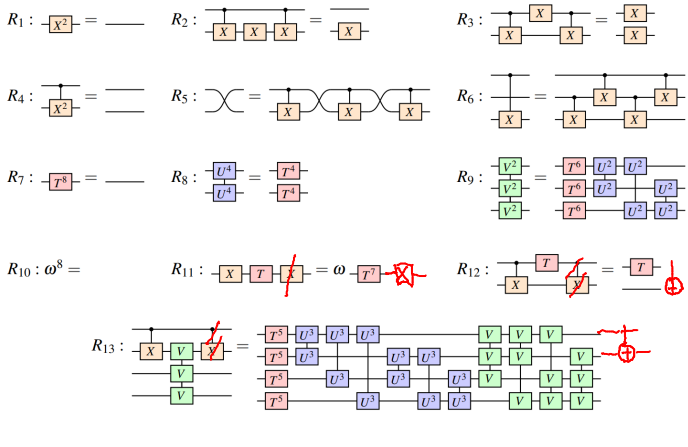


CNOT+Dihedral



where $\mathcal{T} = \{ T^k,\ k = 0, \ldots 7 \}$.

CNOT+Dihedral — Complete relations, 3 parts.



① $GrAn(\mathbb{Z}_2)$ rels

$R_1 - R_6$. 6 rels vs 9 of Lafont.

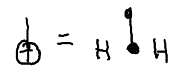
② M rels, $R_7 - R_{10}$

③ semidirect rels

$R_{11} - R_{13}$.

Clifford – Selinger's work.

Clifford circuits are generated by $\{H, S, \oplus\} = \{H, S, \bullet\!\!-\!\!\bullet\}$



We see $\{\oplus, S\} \leq \{\oplus, T\}$

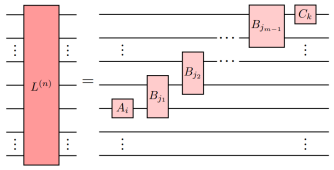
- Trying to first solve $\{\oplus, S\}$ problem like Chen et al did.

  Then solve $\{\oplus, S, H\}$ might be an interesting project.

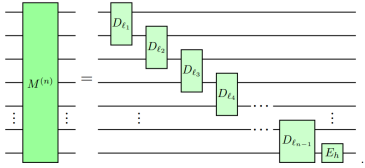
  Selinger did in a different way.

Clifford ~ NFs.

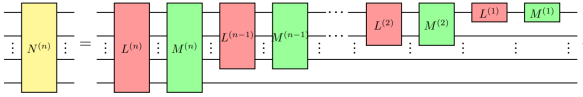
Definition 4.3. We say that an n-qubit circuit is *Z-normal* if it is of the form

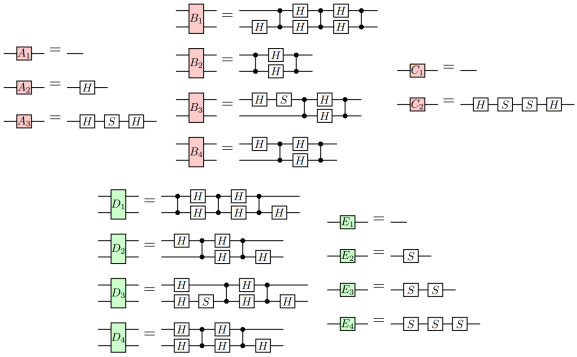


for $1 \leqslant m \leqslant n$. We say that an n-qubit circuit is *X-normal* if it is of the form



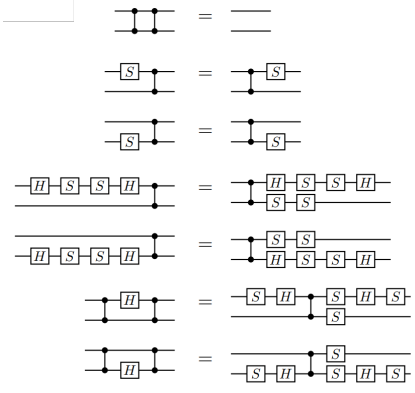
Finally, an n-qubit circuit is *normal* if it is of the form

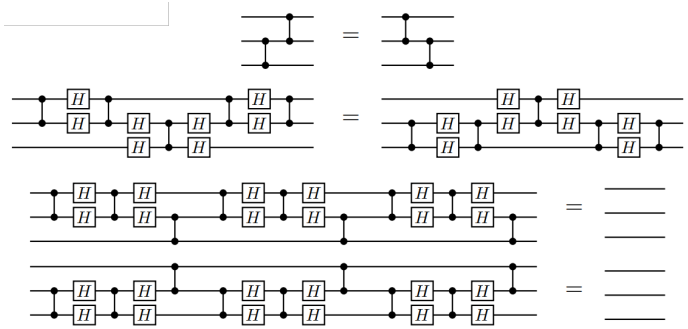




Clifford — Rels.

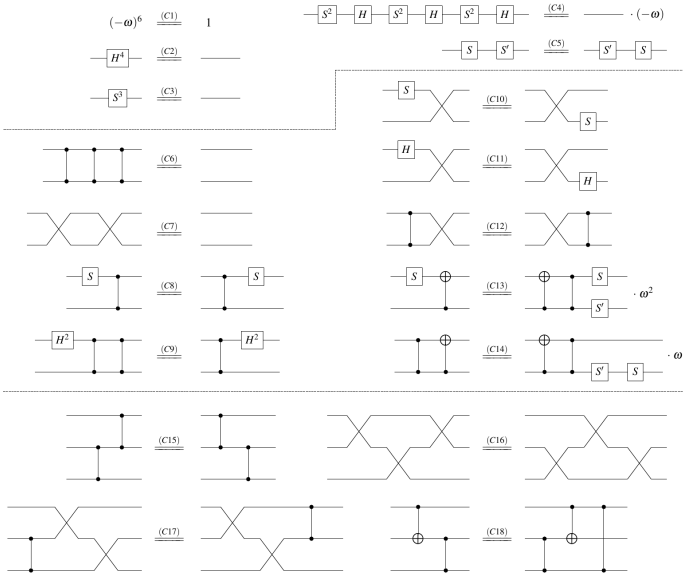
$$H^2 = 1$$
$$S^4 = 1$$
$$SHSHSH = \omega$$





- all vertically symmetric.
- See paper for proofs.
- Other techniques are used.

Qutrit Clifford — NFs and Rels.



- NFs are almost identical.
- We expect rels are also almost identical.
- I further simplified these rels, now they are as expected.
- We are trying to extend the result to qupit.

Current and possible future work

- Current work:
  - Simplify Li et al.'s relations. Almost done. We get simplified relations that are similar to Selinger's which enjoys the "vertical symmetry" property.
  - Normal forms and complete relations for qupit Cifford. Joint work with Li et al.
    - Normal form: done.
    - Collecitng relations and relation simplification: in proress.
- Possible future projects:
  - Formalizing the results introduced today in Agda.
  - Normal forms and complete relations for other classes of circuits, in particular, the classes in honorar y mentions (next slide).

Some honorary mentions

- In 2008, Matsumoto et al. gave normal forms for 1-qubit Clifford+T circuits.

$$(T \mid \varepsilon)\,(HT \mid SHT)^* \mathcal{C}$$

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}, \quad \omega = e^{i\pi/4}.$$

- In 2019, Glaudell et al. gave normal forms for 1-qutrit Clifford+T circuits.

$$\left(\varepsilon \middle| T \middle| H^2 T\right)\left(HT \middle| H^3 T \middle| SHT \middle| SH^3 T \middle| S^2 HT \mid S^2 H^3 T\right)^* \mathcal{C}.$$

- In 2021, Glaudell et al. gave normal forms for 2-qubit Clifford+CS circuits.

$$\bar{U} = \bar{R}_k \cdots \bar{R}_1 \cdot \bar{C}$$

- In 2023, Longcheng Li et al. gave almost-normal forms 2-qubit Clifford+T circuits, similar to 2-qubit Clifford+CS.
- In 2023, Bian et al. gave almost-normal forms and complete relations for 3-qubit Clifford+CS circuits.

$$ce_4e_3e_2e_1ce_4e_3e_2e_1 \ldots ce_4e_3e_2e_1c\overline{CQD}.$$

Demo

- Linear permutation.
- $C_n^n \rtimes S_n$.
- Simplification of Li et al's rels.

```
data X : ℕ -> Set where
  swap : ∀ {n} -> X (suc n)
  _ₛ : ∀ {n} -> X n -> X (suc n)
```

← generators of S_n

```
data rel : (n : ℕ) -> Rel (Word (X n)) 0ℓ where
  order : ∀ {n} ->
    rel (suc n) ([ swap ↑] • [ swap ↑]) ε
  comm : ∀ {n} {a : X n} ->
    rel (2+ n) ([ swap ↑] • [ a ₛ ₛ ↑]) ([ a ₛ ₛ ↑] • [ swap ↑])
  yang-baxter : ∀ {n} ->
    rel (2+ n) ([ swap ↑] • [ swap ₛ ↑] • [ swap ↑]) ([ swap ₛ ↑] • [ swap ↑] • [ swap ₛ ↑])
  congₛ : ∀ {n w v} -> rel n w v ->
    rel (suc n) [ w ↑] [ v ↑]
```

← complete rels of S_n

```
data C : ℕ -> Set where
  ε : ∀ {n} -> C n
  swap•_ : ∀ {n} -> C n -> C (suc n)
```

← C_n

```
NF : ℕ -> Set
NF zero = ⊤
NF (suc n) = NF n × C (suc n)
```

← NF

```
pres-SnD : (k : ℕ) -> WRel (⊤ ⊎^ suc k ⊎ X k)
pres-SnD k = C^n (suc k) ⋊ Sn.rel k ⋆ conj k

nfp : (k : ℕ) -> NFProperty (C^n (suc k) ⋊ Sn.rel k ⋆ conj k)
nfp k = NFP0.nfp (NDP.nfp (Cyclic.pres N) (suc k) (Cyclic.nfp N)) (Sn.nfp k)
```

```
data _===_ : WRel Gen where
  C2-order-H₀ : H₀ ^ 4 === ε
  C3-order-S₀ : S₀ ^ 3 === ε
  C4-order-S₀H₀ : (S₀ • H₀) ^ 3 === ε
  C5-comm-S₀-S₀' : S₀' • S₀ === S₀ • S₀'

  C2-order-H₁ : H₁ ^ 4 === ε
  C3-order-S₁ : S₁ ^ 3 === ε
  C4-order-S₁H₁ : (S₁ • H₁) ^ 3 === ε
  C5-comm-S₁-S₁' : S₁' • S₁ === S₁ • S₁'

  C6-order-CZ : CZ ^ 3 === ε
  C7-order-Ex : Ex ^ 2 === ε
  C8-comm-CZ-S₀ : CZ • S₁ === S₁ • CZ
  C9-semi-CZ-H₀H₀ : CZ • H₀H₀ === H₀H₀ • CZ ^ 2
  C10-semi-EX-S₀ : Ex • S₀ === S₁ • Ex
  C11-semi-EX-H₀ : Ex • H₀ === H₁ • Ex
  C12-comm-EX-CZ : Ex • CZ === CZ • Ex
  C13-semi-XC-S₀ : XC • S₀ === S₀ • S₁' • CZ ^ 2 • XC
  C14-semi-XC-CZ : XC • CZ === S₁ ^ 2 • S₁' ^ 2 • CZ • XC

  -- Monoidal structure rules.
  comm-H₁H₀ : H₁ • H₀ === H₀ • H₁
  comm-H₁S₀ : H₁ • S₀ === S₀ • H₁
  comm-S₁H₀ : S₁ • H₀ === H₀ • S₁
  comm-S₁S₀ : S₁ • S₀ === S₀ • S₁
```

```
data _===_ : WRel Gen where
  order-S₀ : S₀ ^ 3 === ε
  order-H₀ : H₀ ^ 4 === ε
  order-S₀H₀ : (S₀ • H₀) ^ 3 === ε
  comm-H₀H₀S₀H₀H₀S₀ : H₀ • H₀ • S₀ • H₀ • H₀ • S₀ === S₀ • H₀ • H₀ • S₀ • H₀ • H₀

  order-S₁ : S₁ ^ 3 === ε
  order-H₁ : H₁ ^ 4 === ε
  order-S₁H₁ : (S₁ • H₁) ^ 3 === ε
  comm-H₁H₁S₁H₁H₁S₁ : H₁ • H₁ • S₁ • H₁ • H₁ • S₁ === S₁ • H₁ • H₁ • S₁ • H₁ • H₁

  comm-H₁H₀ : H₁ • H₀ === H₀ • H₁
  comm-H₁S₀ : H₁ • S₀ === S₀ • H₁
  comm-S₁H₀ : S₁ • H₀ === H₀ • S₁
  comm-S₁S₀ : S₁ • S₀ === S₀ • S₁

  order-CZ : CZ ^ 3 === ε
  comm-CZ-S₀ : CZ • S₀ === S₀ • CZ
  comm-CZ-S₁ : CZ • S₁ === S₁ • CZ
  comm-CZ-H₀H₀ : CZ • H₀H₀ === H₀H₀ • CZ ^ 2
  comm-CZ-H₁H₁ : CZ • H₁H₁ === H₁H₁ • CZ ^ 2

  rel-CZ-H₀-CZ : CZ • H₀ • CZ === S₀ ^ 2 • H₀ • S₀ ^ 2 • CZ • H₀ • S₀ ^ 2 • S₁ ^ 2 • X₀ ^ 2 • Z₀ ^ 2
  rel-CZ-H₁-CZ : CZ • H₁ • CZ === S₁ ^ 2 • H₁ • S₁ ^ 2 • CZ • H₁ • S₁ ^ 2 • S₀ ^ 2 • X₁ ^ 2 • Z₁ ^ 2

  rel-X₀-CZ : CZ • X₀ === X₀ • Z₁ • CZ
  rel-X₁-CZ : CZ • X₁ === X₁ • Z₀ • CZ
```

```
f-wd-ax : ∀ {w v} -> w ===ₛ v ->  w ≈ v
f-wd-ax C2-order-H₀ = axiom order-H₀
f-wd-ax C3-order-S₀ = axiom order-S₀
f-wd-ax C4-order-S₀H₀ = axiom order-S₀H₀
f-wd-ax C5-comm-S₀-S₀' = by-equal-nf auto
f-wd-ax C2-order-H₁ = by-equal-nf auto
```

Thank you for your attention.

References

M. Amy, J. Chen, and N. J. Ross.
A finite presentation of cnot-dihedral operators.
Electronic Proceedings in Theoretical Computer Science, 266:84–97, Feb. 2018.

Y. Lafont.
Towards an algebraic theory of Boolean circuits.
Journal of Pure and Applied Algebra, 184:257–310, 2003.

S. M. Li, M. Mosca, N. J. Ross, J. van de Wetering, and Y. Zhao.
A complete and natural rule set for multi-qutrit clifford circuits.
Electronic Proceedings in Theoretical Computer Science, 426:23–78, Aug. 2025.

P. Selinger.
Generators and relations for n-qubit Clifford operators.
Logical Methods in Computer Science, 11(2:10):1–17, 2015.
Also available from arXiv:1310.6813.